

Gradient Descent

IIT Madras

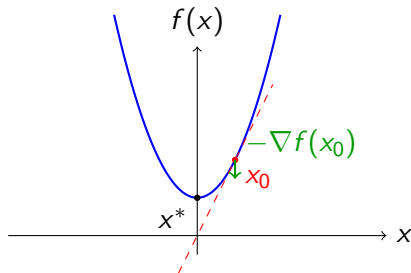
November 17, 2025

First-Order Unconstrained Optimization Problem

Problem Setup:

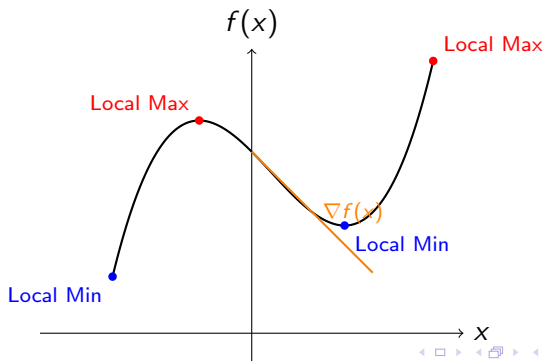
$$\min_{x \in \mathbb{R}^n} f(x)$$

- Assumptions: f is convex and differentiable everywhere.
- Gradient $\nabla f(x)$ points in direction of *steepest increase*.
- To minimize, move in opposite direction $-\nabla f(x)$.
- At the minimum x^* , $\nabla f(x^*) = 0$



Why $\nabla f(x)$ is essential in Minimization Problems

- The gradient $\nabla f(x)$ gives the slope of the curve.
- Local minima occurs when $\nabla f(x) = 0$ and slope changes from negative to positive.
- Local maxima occurs when $\nabla f(x) = 0$ and slope changes from positive to negative.
- In an **unconstrained convex optimization problem**:
 - Any stationary point is the global minimum.



Why is $\nabla f(x) = 0$ in unconstrained optimization?

- Let's say x is a minimizer of the function $f : \mathbb{R}^n \rightarrow \mathbb{R}$.
- Then along the direction $d \in \mathbb{R}^n$, $f(x + td) \geq f(x)$ for all $t \geq 0$
- The directional derivative,

$$\langle \nabla f(x), d \rangle = f'(x; d) = \lim_{t \downarrow 0} \frac{f(x + td) - f(x)}{t} \geq 0,$$

- Thus, $\langle \nabla f(x), d \rangle \geq 0 \quad \forall d \in \mathbb{R}^n$.
- We need to iterate along the descent direction $d = -\nabla f(x)$, leading to

$$-\|\nabla f(x)\|^2 \geq 0 \iff \nabla f(x) = 0.$$

Why is the steepest descent $d = -\nabla f(x)$?

- The gradient $\nabla f(x)$ points in the direction of the **steepest increase**.
- To minimize $\langle \nabla f(x), d \rangle$, we solve

$$\min_{\|d\|=1} \langle \nabla f(x), d \rangle.$$

- By Cauchy–Schwarz:

$$\langle \nabla f(x), d \rangle \geq -\|\nabla f(x)\| \cdot \|d\|,$$

- As $\|d\| = 1$, $\min_{\|d\|=1} \langle \nabla f(x), d \rangle = -\|\nabla f(x)\|$

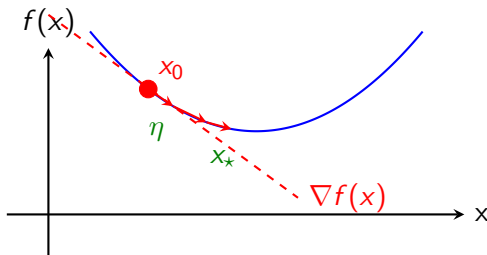
$\Rightarrow$ The minima is attained when $d = -\nabla f(x)$.

Gradient Descent Update Rule

- The idea of gradient descent is to iteratively move along the direction that minimizes $f(x)$.
- Moves $\eta > 0$ distance in the direction $-\nabla f(x_t)$:

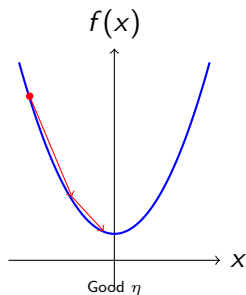
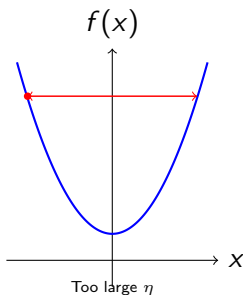
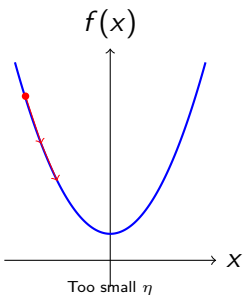
$$x_{t+1} := x_t - \eta \nabla f(x_t) \quad (1)$$

- x_0 is the *initial point*
- $\eta > 0$ the *stepsize* or *learning rate*.



Choosing the Step Size

- The step size η controls how far we move along $-\nabla f(x)$ at each update.
- **Too small:** Converges very slowly.
- **Too large:** Overshoots and misses the local minima.
- **Good step size:** Efficient convergence to the minimum.



Effect of Step Size on Convergence

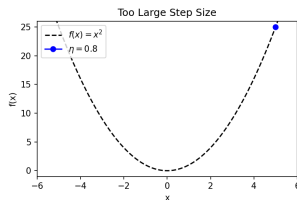
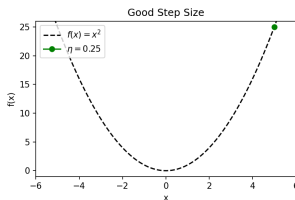
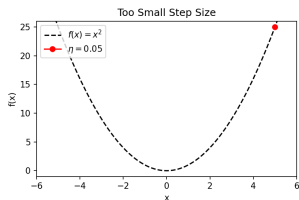
Problem Setup:

- Consider the quadratic function $f(x) = x^2$ with gradient $\nabla f(x) = 2x$.
- Gradient descent update rule:

$$x_{t+1} = x_t - \eta \nabla f(x_t) = (1 - 2\eta)x_t.$$

- Run the gradient descent algorithm for three different η and observe the convergence.

Effect of Step Size on Gradient Descent (Step 0)



Stopping Criteria in Gradient Descent

- Gradient descent is an iterative algorithm: we need a rule to decide when to stop.
- Common stopping criteria:

① **Gradient test:** Stop when we are close to a stationary point:

$$\|\nabla f(x_t)\| < \epsilon.$$

② **change in function value:** Stop when progress has stalled

$$|f(x_{t+1}) - f(x_t)| < \delta.$$

③ **Maximum iterations:** Stop after $T_{\max}$ steps to avoid infinite loops.

Remark: In practice, a combination of these rules is often used.

Gradient Descent Algorithm

Algorithm 1 Gradient Descent

- 1: Choose initial point $x^0 \in \mathbb{R}^n$
 - 2: **for** $t = 1, 2, \dots, T$ **do**
 - 3: Compute the gradient $g = \nabla f(x^t) \in \mathbb{R}^n$
 - 4: Update the point: $x^{t+1} = x^t - \eta_t g$
 - 5: Stop when $\|\nabla f(x^t)\|_2^2 < \epsilon$ for some small $\epsilon > 0$
 - 6: **end for**
-

Source: M. Gormley, "Lecture 7: Gradient Descent", CMU.

Linear Regression using Gradient Descent

We want to fit a line

$$y \approx mx + c$$

to data points (x_i, y_i) by minimizing the least squares loss:

$$L(m, c) = \frac{1}{N} \sum_{i=1}^N (y_i - (mx_i + c))^2$$

Gradients:

$$\frac{\partial L}{\partial m} = -2 \sum_{i=1}^N x_i (y_i - (mx_i + c)), \quad \frac{\partial L}{\partial c} = -2 \sum_{i=1}^N (y_i - (mx_i + c))$$

Suppose data: $(0.5, 1.4), (2.3, 1.9), (2.9, 3.2)$

Start with $m = 1, c = 0$, learning rate $\eta = 0.01$.

Numerical Example

Derivations for the 1st Iteration

$$L(m, c) = \left[(1.4 - (m \cdot 0.5 + c))^2 + (1.9 - (m \cdot 2.3 + c))^2 + (3.2 - (m \cdot 2.9 + c))^2 \right]$$

At $(m, c) = (1, 0)$:

$$\frac{\partial L}{\partial m} = -2(0.5 \cdot (1.4 - 0.5) + 2.3 \cdot (1.9 - 2.3) + 2.9 \cdot (3.2 - 2.9)) = -0.8$$

$$\frac{\partial L}{\partial c} = -2((1.4 - 0.5) + (1.9 - 2.3) + (3.2 - 2.9)) = -1.6$$

Applying the update rule Update:

$$m \leftarrow 1 - 0.01(-0.8) = 1.008, \quad c \leftarrow 0 - 0.01(-1.6) = 0.016$$

So after 1 step: $(m, c) \approx (1.008, 0.016)$

Verification: Try running the code to verify that, (m, c) converges toward the best fit line

$$y \approx 0.64x + 0.95.$$

Gradient Descent Application Example

Objective: Build a regression model from scratch to predict disease progression in diabetes patients. Dataset: `scikit-learn` diabetes dataset.

http://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_diabetes.html.

- Features (X): 10 variables
- Target (y): A quantitative measure of disease progression.
- Model: $h_{\theta}(X) = X \cdot \theta$ where θ is a vector with 11 parameters (10 weights + 1 intercept).

Gradient Descent for Multivariable Regression

Model Setup:

$$J(\theta) = \frac{1}{2N} \sum_{i=1}^N (h_{\theta}(x^{(i)}) - y^{(i)})^2 \quad \theta := \theta - \eta \cdot \frac{1}{N} X^{\top} (X\theta - y)$$

Task:

- 1 Load data: `from sklearn.datasets import load_diabetes`
- 2 Preprocess: add intercept column to X , reshape y
- 3 Implement GD: custom `linear_regression(X,y,...)`
- 4 Train: run until convergence, collect θ and cost history
- 5 Validate: compare with `sklearn.LinearRegression` coefficients

Stochastic Gradient Descent

The Gradient Descent algorithm uses the *entire dataset* for every update. This causes scalability issues in terms of,

- being extremely slow: millions of points processed for one update.
- High memory: dataset may not fit into RAM.

The Solution: Stochastic Gradient Descent (SGD)

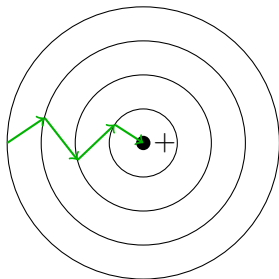
- Approximate the gradient using just *one random data point*.
- Shuffle dataset, loop over samples:

$$\theta := \theta - \eta \nabla J_i(\theta)$$

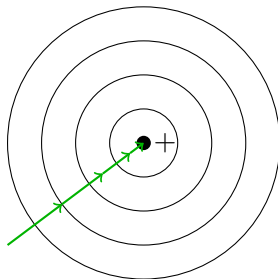
- Fast, lightweight updates $\Rightarrow$ noisy but efficient convergence.

Batch vs Stochastic Gradient Descent

Stochastic Gradient Descent



Gradient Descent



Key Difference:

- In SGD, the updates are made based on a single data point.
- The random selection of data points introduces stochasticity.